Healthcare AI Agentic RAG System

Submitted by: Shlok Kevadiya

Date: 10 April 2026

Architecture Choices

To ensure high accuracy and strictly prevent hallucinations in a healthcare context, this RAG pipeline utilizes a multi-agent, chain-of-thought architecture:

- **Agent 1 (Query Decomposer & Retriever):** Complex questions are often poorly handled by single vector searches. Agent 1 uses a zero-shot prompt to decompose the user's question into three distinct, keyword-rich sub-queries.
- **Vector Database & Embeddings:** I used a local **ChromaDB** paired with HuggingFace's all-MiniLM-L6-v2 embeddings. It is lightweight, fast, and highly effective for document retrieval.
- **MMR Search (Maximal Marginal Relevance):** Instead of standard similarity search, I implemented MMR. This fetches a larger pool of documents (15) but filters them down to the 4 most *diverse* chunks per sub-query. This solved the "context blindness" issue where the database would repeatedly pull from the exact same article while missing complementary facts in other documents.
- **Chunking Strategy:** Documents were split using RecursiveCharacterTextSplitter with a chunk_size of 1000 and a chunk_overlap of 250. This generous overlap ensures that statistics and their surrounding context are never accidentally separated.
- **Agent 2 (Reasoner & Synthesizer):** Powered by **Llama-3.3-70b-versatile** via Groq. I chose this specific model because its massive parameter count provides superior reading comprehension and logical deduction compared to smaller 8B models. It forces a Chain-of-Thought (<thinking>) process before outputting the final cited answer.

Handling Live URL Ingestion

- **Current Implementation:** To ensure the system could answer the breaking news question (Q11), the contents of the live URL were extracted and saved as a text file within the static `data/knowledge_base` directory prior to running the ingestion script.
- **Ideal Dynamic Implementation:** In a fully automated production environment, I would implement a dedicated "Web Scraping Tool" using LangChain's `WebBaseLoader` or `SeleniumURLLoader`. If Agent 1 detected a URL in the prompt, it would trigger this tool to scrape the page text at runtime, chunk it, and append it to a temporary, in-memory vector store so it could be queried alongside the historical Chroma database.

What Worked Well

- **Eradicating Hallucinations:** Shifting the system prompt to an "Evidence-Based Synthesis" approach was a massive success. Instead of panicking over strict rules, the 70B model learned to act like an analyst. It successfully identified industry synonyms (e.g., recognizing that "Clinical Documentation" in the text matched the user's query for "Ambient Notes") while still refusing to answer if the exact statistical data was missing.
- **Citation Accuracy:** The prompt engineering successfully forced the LLM to append the exact source file (e.g., Art_04) to every factual claim, making the outputs highly verifiable for medical professionals.

What I'd Improve with More Time

1. **Advanced Reranking:** While MMR improved diversity, adding a cross-encoder model (like Cohere Rerank) between the retrieval and synthesis steps would push the most factually dense chunks to the very top of the context window, further reducing the LLM's cognitive load.
2. **Conversational Memory Optimization:** Currently, the entire chat history is passed back to the LLMs. With more time, I would implement a buffer memory that summarizes older interactions to save tokens and keep the LLM hyper-focused on the immediate context.
3. **Automated Evaluation:** I would integrate a framework like RAGAS (Retrieval Augmented Generation Assessment) to programmatically score the application's faithfulness and answer relevance across a wider dataset.

Step to run the app(code):

Ensure you have Python 3.9+ installed on your system

my_groq_key = "YOUR_GROQ_API_KEY_HERE" put your Groq API HERE

Run on command prompt:

- **pip install -r requirements.txt**
- **python ingestion.py**
- **streamlit run app.py**